SCALE FOR PROJECT PYTHON MODULE 03 (/PROJECTS/PYTHON-MODULE-03)

You should evaluate 1 student in this team

★

Git repository

```
git@vogsphere-v2.42madrid.com:vogsphere/intra-uuid-ca781815-6775-4a8
```

Introduction

- Remain polite, courteous, respectful and constructive throughout the evaluation process. The well-being of the community depends on it.

- Together with the person (or group) being evaluated, identify any issues in the work. Take the time to discuss and debate the problems you have identified.

- You must consider that there might be differences in how your peers have understood the project's instructions and the scope of its functionality. Always keep an open mind and grade them as honestly as possible. The pedagogy is valid only if peer evaluation is conducted seriously.

Guidelines

- Only grade the work that is in the student's or group's Git repository.

- Double-check that the Git repository belongs to the student or the group. Ensure that the work is for the relevant project and check that "git clone" is run in an empty folder.

- Check carefully that no malicious aliases were used to fool you and make you evaluate something other than the content of the official repository.

- To avoid any surprises, carefully check that both the evaluating and the evaluated students have reviewed any scripts used to facilitate grading.

- If the evaluating student has not completed that particular project yet, it is mandatory for this student to read the entire subject document prior to starting the defence.

- Use the flags available on this scale to signal an empty repository, a non-functioning program, a norm error, cheating, etc. In these cases, the grading is over and the final grade is 0 (or -42 in case of cheating). However, with the exception of cheating, you are encouraged to continue discussing your work (even if you have not finished it) in order to identify any issues that may have caused this failure and to avoid repeating the same mistake in the future.

- Remember that for the duration of the defence, no unexpected, premature, uncontrolled termination of the program is allowed;

otherwise, the final grade is 0. Use the appropriate flag.
You should never have to edit any file except the configuration file if it exists.
If you want to edit a file, take the time to explain the reasons to the
evaluated student and make sure both of you agree on this change.

Attachments

subject.pdf (https://cdn.intra.42.fr/pdf/pdf/202390/en.subject.pdf)

Preliminaries

Basics

- Only grade the work that is in the learner's Git repository.
- Check that the Git repository belongs to the learner, and that the work is the learner's.
- Check that only the requested files are present at the root of the repository.
- Each exercise must be graded in the order of the subject.
- Check that no malicious aliases are used.
- Verify that the project structure matches the subject requirements.

Are all preliminary checks satisfied?

Yes No

Exercises

Exercise 0 - Command quest

Check that the file ft_command_quest.py exists and demonstrates command-line argument processing using lists:

- The program correctly accesses sys.argv
- It displays the program name (sys.argv[0])
- It counts and displays the total number of arguments
- It handles cases with no arguments provided
- It displays each argument individually when provided
- Test with: python3 ft_command_quest.py
- Test with: python3 ft_command_quest.py hello world 42
- Test with: python3 ft_command_quest.py "Data Quest"
- Which strategy has been put in motion to not print the program name along with the arguments? What could be alternatives?

Yes No

Exercise 1 - Score Cruncher

Check that the file ft_score_analytics.py exists and contains the required functionality:

- The program accepts command-line arguments as player scores
- It processes lists of scores correctly
- It handles invalid inputs gracefully (non-numeric arguments, no valid score to process)
- It displays analytics including the total, average, highest and lowest scores, and the range
- Test with: python3 ft_score_analytics.py 100 250 180 90 300
- Test with invalid input: python3 ft_score_analytics.py 100 abc 200
- Test with no arguments: python3 ft_score_analytics.py

Does the program work correctly in all test cases?

Yes No

Exercise 2 - Position Tracker

Check that the file ft_coordinate_system.py exists and demonstrates tuple operations:

- The program asks for coordinates and handles invalid inputs
- It creates and manipulates 3D coordinate tuples (x, y, z)
- It demonstrates tuple unpacking (direct access using [], or x, y, z = coordinate)
- It calculates 3D Euclidean distances correctly
- Test coordinate creation and unpacking
- Test distance calculations between 3D coordinates
- Test error handling with invalid coordinates

Does the program correctly demonstrate tuple concepts for 3D coordinates?

☑ Yes ☒ No

Exercise 3 - Achievement Hunter

Check that the file ft_achievement_tracker.py exists and uses sets effectively:

- The gen_player() function creates and returns a set of random achievements
- The program finds the set of all players' achievements (union)
- It finds common achievements between players (intersection)
- It identifies unique achievements per player (difference)
- It reports the missing achievements per player (difference)
- Run the program multiple times to check that sets are generated randomly

Ask the learner to explain set operations (union, intersection, difference) and their use cases.

☑ Yes ☒ No

Exercise 4 - Inventory Master

Check that the file ft_inventory_system.py exists and uses dictionaries comprehensively:

- The program accepts command-line arguments in the format item:quantity
- It correctly handles invalid parameters
- It stores inventory data in a dictionary structure (item names as keys, quantities as values)
- It displays the items without their quantities
- It calculates inventory statistics (total items and quantity, percentages, most/least abundant)
- It uses dictionary methods: keys(), values(), update() (alternate solutions are allowed)
- Test with: python3 ft_inventory_system.py sword:1 potion:5 shield:2 armor:3
- Test with no arguments to verify that a usage message is displayed
- Test with an invalid format to verify that error handling works correctly

Ask the learner to explain dictionary operations.
Does the program effectively use dictionaries for inventory management?

☑ Yes ☒ No

Exercise 5 - Stream Wizard

Check that the file ft_data_stream.py exists and demonstrates generators:

- The program creates the generator function gen_event() that uses yield and provides an event tuple
- It uses next() to get the next event tuple inside a "for" loop that runs 1000 times
- It prints the 1000 events
- Verify that the program does not create a list (or other structure) of 1000 elements
- It then builds a list of 10 tuples generated by gen_event()
- The program creates the generator function consume_event(), which uses yield to provide a random remaining element of the list while removing it
- Check that consume_event() is turned into an iterable and used in a "for .. in .." construct and not inside the "for" loop. You should have something like "for e in consume_ev:" with consume_ev being the generator for consume_event().

Ask the learner to explain how generators work and why they're memory efficient.

☑ Yes ☒ No

Exercise 6 - Data Alchemist

Check that the file ft_data_alchemist.py exists and uses all types of comprehensions:

- The program creates a list of names and displays it
- It uses list comprehensions for the capitalized list (all names) and for the sub-list of already-capitalized names
- The program creates a dictionary using a comprehension, based on the previous names and random scores
- It uses another dictionary comprehension to create the dict of high scores
- Run the program multiple times to get different scores, and update the list of players to check that the program's behavior is correct

Does the program demonstrate mastery of comprehensions for data processing?

☑ Yes ☒ No

General Review

Code Quality

Assess the overall code quality:

- Is the code written in Python 3.10 or higher?
- Does the code adhere to the flake8 linter standards (no errors)?
- Type hints are REQUIRED for all functions and methods (parameters and return values). Check this using mypy.
- Docstrings are NOT required for this module.
- Is the code well-structured and readable?
- Is error handling implemented appropriately?

☑ Yes ☒ No

Ratings

Don’t forget to check the flag corresponding to the defense

✔ Ok ★ Outstanding project

Empty work Incomplete work Invalid compilation Norme Cheat Crash

Concerning situation Leaks Forbidden function Can’t support / explain code

Conclusion

Leave a comment on this evaluation (2048 chars max)

Finish evaluation

API General Terms of Use (https://profile.intra.42.fr/legal/terms/33) Declaration on the use of cookies (https://profile.intra.42.fr/legal/terms/2) Privacy policy (https://profile.intra.42.fr/legal/terms/5) General term of use of the site (https://profile.intra.42.fr/legal/terms/6) Rules of procedure (https://profile.intra.42.fr/legal/terms/4) Terms of use for video surveillance (https://profile.intra.42.fr/ Legal notices (https://profile.intra.42.fr/legal/terms/3)

legal/terms/1)